

Технологии анализа больших данных

Экзамен (Apache Spark) — правильные ответы с аргументацией. 60 вопросов.

Сводная таблица ответов

1 - B	2 - E	3 - E	4 - D	5 - E	6 - A	7 - A	8 - E	9 - E	10 - D
11 - A	12 - E	13 - A	14 - A	15 - D	16 - D	17 - C	18 - C	19 - A	20 - B
21 - A	22 - A	23 - D	24 - C	25 - B	26 - C	27 - A	28 - D	29 - E	30 - E
31 - A	32 - A	33 - D	34 - E	35 - B	36 - A	37 - E	38 - B	39 - D	40 - D
41 - B	42 - A	43 - C	44 - B	45 - D	46 - D	47 - C	48 - A	49 - B	50 - B
51 - B	52 - A	53 - E	54 - A	55 - E	56 - E	57 - E	58 - C	59 - D	60 - E

Ниже — подробный разбор каждого вопроса.

Вопрос 1. Какое утверждение о драйвере Spark НЕВЕРНО?

✓ **Правильный ответ: В. Драйвер Spark горизонтально масштабируется для увеличения пропускной способности.**

Драйвер — единственный, неразмножаемый процесс приложения: он не масштабируется горизонтально. Горизонтально масштабируются только рабочие узлы и исполнители. Остальные утверждения (А, С, D, E) верны, поэтому ложным (а значит искомым) является В.

Вопрос 2. Что описывает узлы в Spark в кластерном режиме?

✓ **Правильный ответ: Е. Рабочие узлы — это машины, на которых размещаются исполнители, отвечающие за выполнение задач.**

Узел (node) — это физическая/виртуальная машина. Рабочие узлы host исполнителей. А неверно (самый детальный уровень — задача/слот), В описывает локальный режим, С путает узлы с исполнителями, D неверно — драйвер не масштабируется горизонтально.

Вопрос 3. Какое утверждение о слотах верно?

✓ **Правильный ответ: Е. Слоты — это ресурсы для параллелизма внутри приложения Spark.**

Слот — это ресурс для выполнения одной задачи параллельно (обычно = ядро). Связи «слотов больше исполнителей» (А) или «задач больше слотов» (В) не являются обязательными правилами; самый детальный уровень — задача, а не слот (С); слоты используются во всех режимах (D неверно).

Вопрос 4. Комбинация блока данных и набора трансформаций на одном исполнителе?

✓ **Правильный ответ: D. Задача (Task).**

Задача = один раздел данных + цепочка трансформаций, выполняемая на одном исполнителе. Это самая мелкая единица выполнения. Исполнитель — это процесс, узел — машина, стадия — группа задач, задание — реакция на действие.

Вопрос 5. Группа задач, выполняемых параллельно для одного набора операций?

✓ **Правильный ответ: Е. Стадия (Stage).**

Стадия — это набор задач, которые можно выполнять параллельно над одним и тем же набором операций (между перемешиваниями). Задание (Job) запускается действием и делится на стадии; стадии делятся на задачи.

Вопрос 6. Что описывает перемешивание (shuffle)?

✓ **Правильный ответ: А. Перемешивание — это процесс перераспределения данных между разделами.**

Shuffle — перераспределение данных между разделами (часто с передачей по сети между исполнителями), которое требуется широким преобразованиям. Формулировка через разделы (partitions) — точное и каноническое определение.

Вопрос 7. df очень большой, разделов больше, чем исполнителей. Что НЕВЕРНО?

✓ **Правильный ответ: А. Производительность будет ниже оптимальной, потому что не все исполнители будут использоваться одновременно.**

Когда разделов БОЛЬШЕ, чем исполнителей, все исполнители как раз будут загружены постоянно — простаивающих нет. Поэтому это утверждение ложно (искомое). Остальные пункты (накладные расходы, очередь обработки, риск OOM) корректны.

Вопрос 8. Какая операция является действием (action)?

✓ **Правильный ответ: Е. DataFrame.count()**

count() запускает вычисление и возвращает значение драйверу — это действие. filter(), distinct(), intersect(), join() ленивы и возвращают новый DataFrame, то есть являются преобразованиями.

Вопрос 9. Разница между преобразованиями и действиями?

✓ **Правильный ответ: Е. Преобразования — бизнес-логика без запуска вычислений; действия — триггеры выполнения, возвращающие результат.**

Преобразования ленивы (строят план), действия материализуют вычисления и возвращают результат. Вариант С переставляет понятия местами, поэтому верен именно Е.

Вопрос 10. Какая операция ВСЕГДА является узким преобразованием?

✓ **Правильный ответ: D. DataFrame.select()**

select() обрабатывает каждый раздел независимо, без перемещения данных между разделами — узкое преобразование. sort(), distinct(), repartition(), join() требуют перемешивания — это широкие преобразования.

Вопрос 11. Что описывает режим выполнения/развертывания Spark?

✓ **Правильный ответ: А. Определяет, где физически расположены драйвер и исполнители при запуске приложения.**

Режим развертывания (cluster/client/local) задаёт физическое расположение драйвера и исполнителей. Назначение задач исполнителям — это работа планировщика, а не режима развертывания.

Вопрос 12. Какая конфигурация обеспечит завершение приложения при отказе рабочего узла?

✓ **Правильный ответ: Е. Сценарий №6 (8 узлов).**

Отказоустойчивость Spark основана на пересчёте утраченных разделов по линии происхождения (lineage). Для этого нужны ОСТАВШИЕСЯ узлы. В сценарии №1 (1 узел) потеря узла = крах приложения. Чем больше узлов, тем выше шанс, что при отказе одного оставшиеся пересчитают потерянные данные — поэтому конфигурация с 8 узлами (№6) наиболее устойчива.

Вопрос 13. Что описывает ошибки нехватки памяти (OOM) в Spark?

✓ **Правильный ответ: А. OOM возникает, когда драйверу или исполнителю не хватает памяти для сбора/обработки выделенных данных.**

Классический OOM: исполнителю не хватает памяти на раздел, или драйверу — на собранный результат (например, при collect()). Остальные варианты описывают несуществующие или искажённые механизмы.

Вопрос 14. Уровень хранения по умолчанию для persist() (не-поточковый DataFrame)?

✓ **Правильный ответ: А. MEMORY_AND_DISK.**

Для DataFrame/Dataset (в отличие от RDD, где по умолчанию MEMORY_ONLY) и persist(), и cache() используют уровень MEMORY_AND_DISK: данные хранятся в памяти, а излишки сбрасываются на диск.

Вопрос 15. Что описывает широковещательную переменную (broadcast)?

✓ **Правильный ответ: D. Полностью кэшируется на каждом рабочем узле, поэтому её не нужно передавать/перемешивать на каждом этапе.**

Broadcast-переменная рассылается один раз и кэшируется целиком на каждом узле, экономя сеть. Её можно создавать и неявно (авто-broadcast при join), а не только явным broadcast() — поэтому В неверно.

Вопрос 16. Какая операция с наибольшей вероятностью вызовет перекос (skew) разделов?

✓ **Правильный ответ: D. DataFrame.coalesce(n)**

coalesce() уменьшает число разделов, объединяя существующие БЕЗ полного перемешивания, из-за чего размеры разделов могут стать неравномерными (skew). repartition() делает полное перемешивание и распределяет данные равномерно.

Вопрос 17. На каких структурах данных построены Spark DataFrames?

✓ **Правильный ответ: C. RDD (Resilient Distributed Datasets).**

DataFrame — это высокоуровневая абстракция поверх RDD: под капотом он реализован как RDD объектов Row со схемой.

Вопрос 18. Вернуть DataFrame только со столбцами storeId и division?

✓ **Правильный ответ: C. storesDF.select("storeId", "division")**

select() принимает имена столбцов как отдельные строковые аргументы. В В/Е имена без кавычек (ошибка), в D неправильный вызов col() с двумя именами, в А два отдельных select() оставят только division.

Вопрос 19. Вернуть все столбцы, КРОМЕ sqft и customerSatisfaction?

✓ **Правильный ответ: A. storesDF.drop("sqft", "customerSatisfaction")**

drop() принимает имена удаляемых столбцов в кавычках. В перечисляет оставшиеся столбцы (хрупко и не соответствует заданию), С/D/E используют имена без кавычек или некорректный синтаксис.

Вопрос 20. Ошибка в storesDF.filter(sqft <= 25000)?

✓ **Правильный ответ: B. Имя нужно взять в кавычки и обернуть в col(): storesDF.filter(col("sqft") <= 25000).**

sqft без кавычек — неопределённая переменная Python. Нужен объект Column: col("sqft"). Вариант Е передаёт col(sqft) (тоже без кавычек) — это останется ошибкой. Знак сравнения менять не требуется.

Вопрос 21. Заполнить пропуски: sqft <= 25000 ИЛИ customerSatisfaction >= 30

✓ **Правильный ответ: A. filter, (col("sqft") <= 25000), |, (col("customerSatisfaction") >= 30)**

В PySpark логическое ИЛИ — оператор |, а каждое условие обязательно берётся в скобки из-за приоритета операций. Ключевое слово or (D, E) не работает с объектами Column; в С нет скобок вокруг условий.

Вопрос 22. Операция для преобразования типа столбца?

✓ **Правильный ответ: A. col().cast()**

Приведение типа выполняется методом cast() объекта Column: col("x").cast("integer"). Функций convert(), castAs(), coerce() в API нет.

Вопрос 23. Новый столбец sqft100 = 1/100 от sqft?

✓ **Правильный ответ: D. storesDF.withColumn("sqft100", col("sqft") / 100)**

withColumn(имя, выражение): имя нового столбца — строка, значение — выражение над Column. А умножает на 100 (не делит), В/Е используют sqft без col(), С ошибочно оборачивает имя в col().

Вопрос 24. Новый столбец numberOfManagers с константой 1?

✓ **Правильный ответ: C. storesDF.withColumn("numberOfManagers", lit(1))**

Литеральное (константное) значение задаётся функцией lit(). Просто 1 (B) не является Column; lit("1") (D) даст строку, а не целое; col(1) и IntegerType(1) синтаксически неверны.

Вопрос 25. Ошибка при разбиении storeCategory по '_' через split()?

✓ **Правильный ответ: B. split()** — функция из модуля `functions`; принимает `Column` и разделитель, не является методом `Column`.

Правильно: `split(col("storeCategory"), "_")`. В коде `split()` вызван как метод `col("...").split("_")`, чего в API нет. Индексация результата `[0]/[1]` как раз допустима.

Вопрос 26. Операция для разбиения столбца-массива на отдельные строки?

✓ **Правильный ответ: C. explode()**

`explode()` создаёт по одной строке на каждый элемент массива. `split()` режет строку в массив, `arrays_zip()` объединяет массивы.

Вопрос 27. storeCategory в нижнем регистре?

✓ **Правильный ответ: A. storesDF.withColumn("storeCategory", lower(col("storeCategory")))**

Функция `lower()` из модуля `functions` принимает `Column`: `lower(col("storeCategory"))`. `lower()` — не метод `Column` (B), функции `tolower()` нет (C), а в E имя без кавычек.

Вопрос 28. Ошибка при переименовании division→state и managerName→managerFullName?

✓ **Правильный ответ: D. Первый аргумент withColumnRenamed() — старое имя, второй — новое.**

Сигнатура: `withColumnRenamed(existing, new)`. В коде аргументы переставлены местами ("state", "division" вместо "division", "state"). Двойной вызов и сами имена корректны — дело именно в порядке аргументов.

Вопрос 29. Удалить строки с пропусками во ВСЕХ столбцах (all)?

✓ **Правильный ответ: E. storesDF.na.drop("all")**

`na.drop("how")` с `how="all"` удаляет строку, только если ВСЕ её значения пустые. `na.drop()` (D) по умолчанию `how="any"` — удалит строку с любым пропуском. `nadrop` (A) и `subset="sqft"` (B) не соответствуют заданию.

Вопрос 30. Какая операция НЕ вернёт DataFrame с уникальными строками?

✓ **Правильный ответ: E. DataFrame.drop_duplicates(subset = "all")**

`subset` ожидает список имён столбцов. "all" будет воспринят как имя несуществующего столбца (или перебор по символам) — корректного удаления дубликатов не произойдёт. `distinct()`, `dropDuplicates()`, `drop_duplicates()` (с `None` или без аргументов) работают.

Вопрос 31. Какой блок НЕ всегда вернёт точное число различных значений division?

✓ **Правильный ответ: A. storesDF.agg(approx_count_distinct(col("division")).alias("divisionDistinct"))**

`approx_count_distinct` по умолчанию использует приближённый алгоритм (HyperLogLog) с погрешностью ~5%, поэтому точное значение не гарантировано. `countDistinct()`, `distinct().count()` и `dropDuplicates().count()` дают точный результат.

Вопрос 32. Новый DataFrame со средним sqft в столбце sqftMean?

✓ **Правильный ответ: A. agg, mean, col("sqft")**

Агрегат среднего: `agg(mean(col("sqft")).alias("sqftMean"))`. `withColumn` (C) не агрегирует, `average` (E) такой функции нет, `mean` как самостоятельный метод `DataFrame` (B) не используется.

Вопрос 33. Вернуть количество строк в storesDF?

✓ **Правильный ответ: D. storesDF.count()**

`count()` на `DataFrame` — действие, возвращающее число строк. `withColumn("...", count())` и `agg(count())` без `groupBy/alias` не дают искомого числа строк.

Вопрос 34. Сумма sqft, сгруппированная по division?

✓ **Правильный ответ: E. storesDF.groupBy("division").agg(sum(col("sqft")))**

Правильная цепочка: groupBy("division") возвращает GroupedData, к которому применяется agg(sum(col("sqft"))). groupBy без аргумента (A) и groupby как свойство (D) неверны.

Вопрос 35. Сводная статистика только для столбца sqft?

✓ **Правильный ответ: B. storesDF.describe("sqft")**

describe("sqft") считает count/mean/stddev/min/max для указанного столбца. summary(col(...)) и describeColumn() — некорректный синтаксис; summary() без аргумента берёт все столбцы.

Вопрос 36. Какие операции сортируют строки DataFrame?

✓ **Правильный ответ: A. sort() и orderBy()**

В API DataFrame есть оба метода — sort() и orderBy() (они синонимы). Имена orderby (нижний регистр) не существуют.

Вопрос 37. Ошибка: 15% выборка БЕЗ замены: sample(True, fraction=0.15)?

✓ **Правильный ответ: E. Первый аргумент True задаёт выборку С заменой.**

Сигнатура: sample(withReplacement, fraction, seed). True означает выборку С возвращением; для выборки без замены нужен False. Параметр seed необязателен, так что проблема именно в первом аргументе.

Вопрос 38. Вернуть первые n строк DataFrame?

✓ **Правильный ответ: B. DataFrame.take(n)**

take(n) возвращает первые n строк (список Row). show(n) печатает, но не возвращает; head без n даёт одну строку; collect(n) такой сигнатуры нет; метода n() не существует.

Вопрос 39. Извлечь значение sqft из первой строки?

✓ **Правильный ответ: D. storesDF.first().sqft**

first() — действие, возвращающее объект Row; к его полю обращаются как к атрибуту: .sqft. first без скобок (A,B,C) — это сам метод, а не результат; col("sqft") к Row не применяется.

Вопрос 40. Что выводит схему DataFrame?

✓ **Правильный ответ: D. DataFrame.printSchema()**

printSchema() печатает древовидную структуру схемы. schema — это свойство (без скобок), оно возвращает объект, но не печатает; schema() со скобками — ошибка; print(storesDF) выводит лишь сигнатуру столбцов.

Вопрос 41. Порядок: создать и применить SQL UDF 'ASSESS_PERFORMANCE'?

✓ **Правильный ответ: B. 1, 4**

Сначала регистрируем UDF: spark.udf.register("ASSESS_PERFORMANCE", assessPerformance) — порядок аргументов (имя, функция). Затем вызываем через SQL по зарегистрированному имени ASSESS_PERFORMANCE (строка 4).

Вопрос 42. Порядок: создать Python UDF и применить к столбцу?

✓ **Правильный ответ: A. 3, 4**

udf(assessPerformance, IntegerType()) — тип возврата создаётся как ЭКЗЕМПЛЯР со скобками (строка 3, в отличие от строки 1). Затем применяем UDF в withColumn (строка 4). Строка 5 вызывает обычную функцию, а не UDF.

Вопрос 43. Операция для выполнения SQL-запроса к таблице?

✓ **Правильный ответ: C. spark.sql()**

spark.sql("...") выполняет SQL-запрос и возвращает DataFrame.

createOrReplaceTempView()/createTempView() лишь регистрируют таблицу, но сами запрос не выполняют.

Вопрос 44. Одностолбцовый DataFrame из списка целых years?

✓ **Правильный ответ: B. `spark.createDataFrame(years, IntegerType())`**

`createDataFrame(данные, схема)`: для списка целых нужна схема `IntegerType()` как ЭКЗЕМПЛЯР (со скобками). Без скобок (E) — передаётся класс, а не тип; без схемы (D) Spark не выведет тип из простого списка чисел.

Вопрос 45. Кэшировать DataFrame только в памяти (значения по умолчанию можно менять)?

✓ **Правильный ответ: D. `DataFrame.persist()`**

`persist()` принимает `StorageLevel`, например `MEMORY_ONLY`, что позволяет хранить данные ТОЛЬКО в памяти. `cache()` жёстко использует `MEMORY_AND_DISK` и уровень не настраивается.

Вопрос 46. Ошибка: 4-разделовый DataFrame из 8 БЕЗ перемешивания: `repartition(4)`?

✓ **Правильный ответ: D. `repartition` вызвал полное перемешивание; нужно использовать `coalesce()`.**

`repartition()` всегда делает полный shuffle. Для УМЕНЬШЕНИЯ числа разделов без перемешивания применяют `coalesce(4)`, который просто объединяет существующие разделы.

Вопрос 47. Что ВСЕГДА вернёт 12-разделовый DataFrame из 8?

✓ **Правильный ответ: C. `storesDF.repartition(12)`**

Увеличить число разделов можно только `repartition()` (он делает полный shuffle). `coalesce()` умеет лишь уменьшать число разделов и не сможет создать 12 из 8.

Вопрос 48. Свойство для числа разделов в широких преобразованиях (join)?

✓ **Правильный ответ: A. `spark.sql.shuffle.partitions`**

`spark.sql.shuffle.partitions` (по умолчанию 200) задаёт число разделов после перемешивания в операциях SQL/DataFrame, таких как `join` и `groupBy`. `spark.default.parallelism` относится к RDD.

Вопрос 49. Порядок: столбец `openDateString` из int-эпохи в строку даты?

✓ **Правильный ответ: B. 2, 1**

Сначала задаём корректный шаблон формата (строка 2: "EEEE, MMM d, yyyy h:mm a"). Затем из UNIX-времени получаем строку через `from_unixtime(col("openDate"), simpleDateFormat)` (строка 1). `date_format` ожидает дату/время, а не целое.

Вопрос 50. Столбец `month` (число месяца) из `openDate` (int-эпоха)?

✓ **Правильный ответ: B. `cast("Timestamp")`, затем `month(col("openTimestamp"))`**

Функция `month()` работает с типами `date/timestamp`, а не с целым. Поэтому сначала приводим `openDate` к `Timestamp` (`col("openDate").cast("Timestamp")`), и только потом извлекаем месяц через `month()`.

Вопрос 51. Какая операция выполняет ВНУТРЕННИЙ join двух DataFrames?

✓ **Правильный ответ: B. `DataFrame.join()`**

`join()` по умолчанию выполняет `inner join` (`how="inner"`). `innerJoin()` и `merge()` в Spark DataFrame API не существуют; `crossJoin()` даёт декартово произведение.

Вопрос 52. Внешнее (outer) соединение по столбцу `storeId`?

✓ **Правильный ответ: A. `storesDF.join(employeesDF, "storeId", "outer")`**

Сигнатура: `join(other, on, how)`. Здесь `on="storeId"`, `how="outer"`. В остальных вариантах аргументы переставлены или используется несуществующий `merge()`.

Вопрос 53. Ошибка: `inner join` по `storeId` И `employeeId` (в обоих DataFrame)?

✓ **Правильный ответ: E. Ссылки не должны быть внутри `col()`; `join` по списку имён `["storeId","employeeId"]` работает.**

При соединении по столбцам с одинаковыми именами в обоих DataFrame передаётся список СТРОК-имён: ["storeld","employeeeld"]. Обёртка col() в этом сокращённом equi-join не подходит — её нужно убрать.

Вопрос 54. Свойство для авто-трансляции DataFrame без broadcast()?

✓ **Правильный ответ: A. spark.sql.autoBroadcastJoinThreshold**

Если размер таблицы меньше autoBroadcastJoinThreshold (по умолчанию 10 МБ), Spark автоматически выполняет broadcast join без явного вызова broadcast(). broadcastTimeout задаёт лишь тайм-аут.

Вопрос 55. Заполнить пропуски для кросс-соединения (cross join)?

✓ **Правильный ответ: E. storesDF.crossJoin(employeesDF)**

Декартово произведение даёт метод crossJoin(other) — без условия и без ключа соединения. Передавать "storeld" или "cross" в него не нужно.

Вопрос 56. Операция поэлементного объединения (union) двух DataFrames?

✓ **Правильный ответ: E. DataFrame.union()**

union() объединяет строки двух DataFrame по позиции столбцов. Это метод DataFrame, а не отдельная функция. unionByName() объединяет по именам столбцов, но вопрос про поэлементное (по позиции) объединение.

Вопрос 57. Записать storesDF в parquet по пути filePath?

✓ **Правильный ответ: E. storesDF.write.parquet(filePath)**

write — это свойство (без скобок), возвращающее DataFrameWriter, у которого есть метод parquet(path). write() со скобками (C) и метода path() (A,B) не существует.

Вопрос 58. Ошибка: запись parquet с разбиением по division?

✓ **Правильный ответ: C. У DataFrameWriter нет repartition(); нужно partitionBy().**

Разбиение при ЗАПИСИ задаётся методом partitionBy("division") у DataFrameWriter. repartition() — это метод DataFrame (меняет число разделов в памяти), а не writer'a, поэтому код упадёт с ошибкой.

Вопрос 59. Прочитать parquet-файл по пути filePath в DataFrame?

✓ **Правильный ответ: D. spark.read.parquet(filePath)**

read — свойство (без скобок), возвращающее DataFrameReader, у которого есть метод parquet(path). read() со скобками и метод path() — неверны.

Вопрос 60. Прочитать JSON по filePath с заданной схемой schema?

✓ **Правильный ответ: E. spark.read.schema(schema).format("json").load(filePath)**

read — свойство (без скобок); schema принимает ОБЪЕКТ схемы (переменную schema, а не строку "schema"); format("json") — в кавычках; load(filePath) загружает данные.